RISC-V Processor

Instructor: Jeongseob Ahn (안정섭)

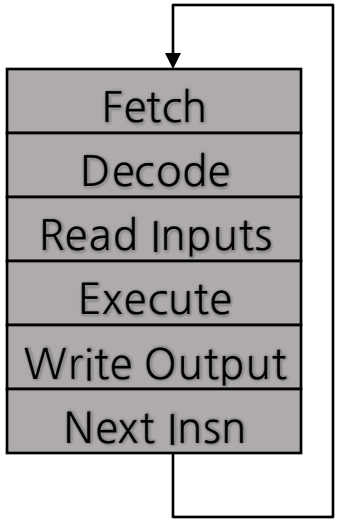
E-mail: jsahn@csl.korea.ac.kr
Hompage: https://jeongseob.github.io

Introduction

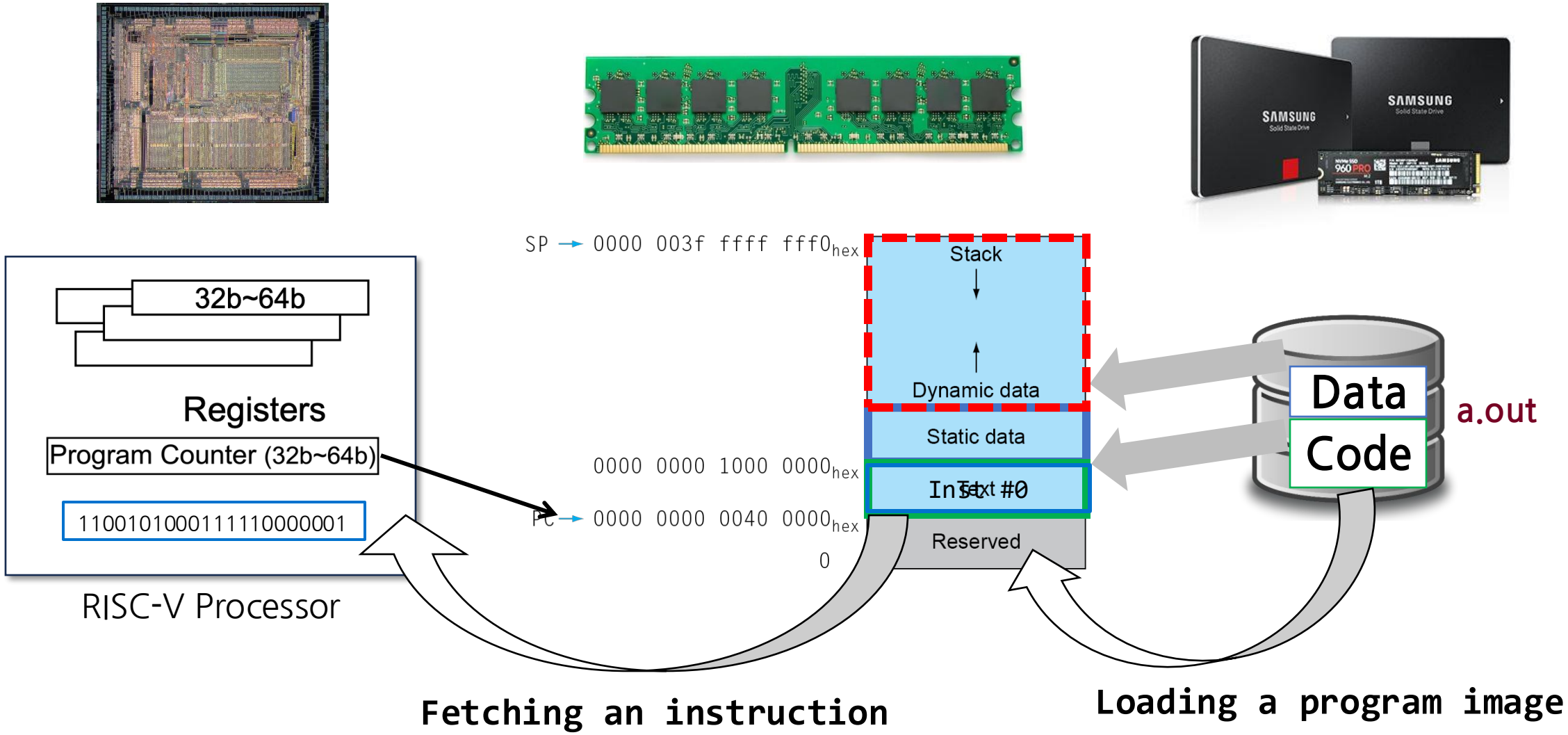
- CPU performance factors
  - Instruction count
    - Determined by ISA and compiler
  - CPI(Cycle Per Instruction) and Cycle time
    - Determined by CPU hardware
- We will study two RISC-V implementations
  - A simplified version
  - A more realistic `pipelined` version
- Simple subset, shows most aspects
  - Memory reference: `ld, sd`
  - Arithmetic/logical: `add, sub, and, or`
  - Control transfer: `beq`

Instruction Execution

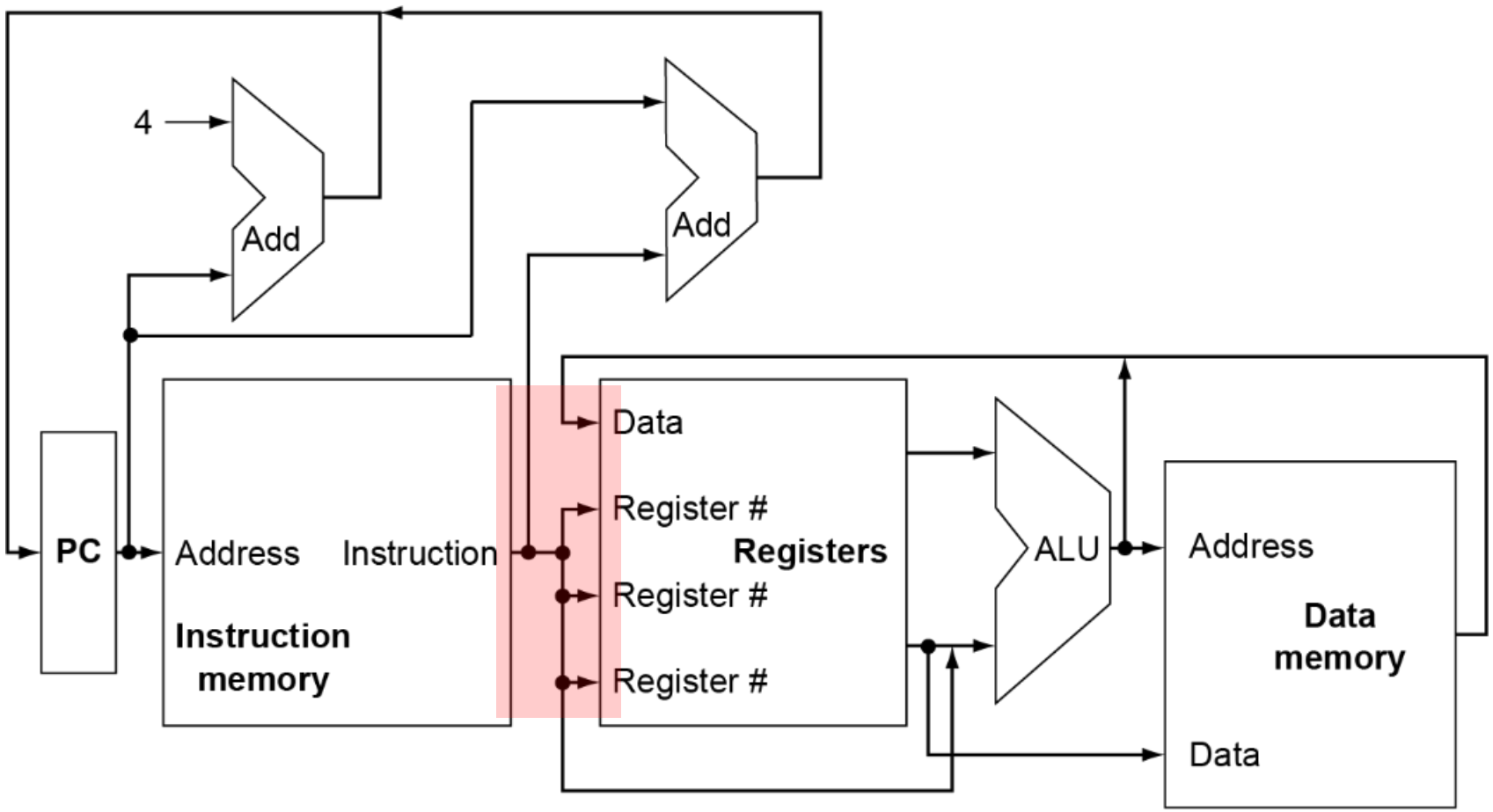
- Fetch instruction
  - PC → instruction memory
- Decode instruction
- Read registers
  - Register numbers → register file
- Depending on instruction class
  - Use ALU to calculate
    - Arithmetic result
    - Memory address for load/store
    - Branch target address
  - Access data memory for load/store
  - PC ← target address or PC + 4



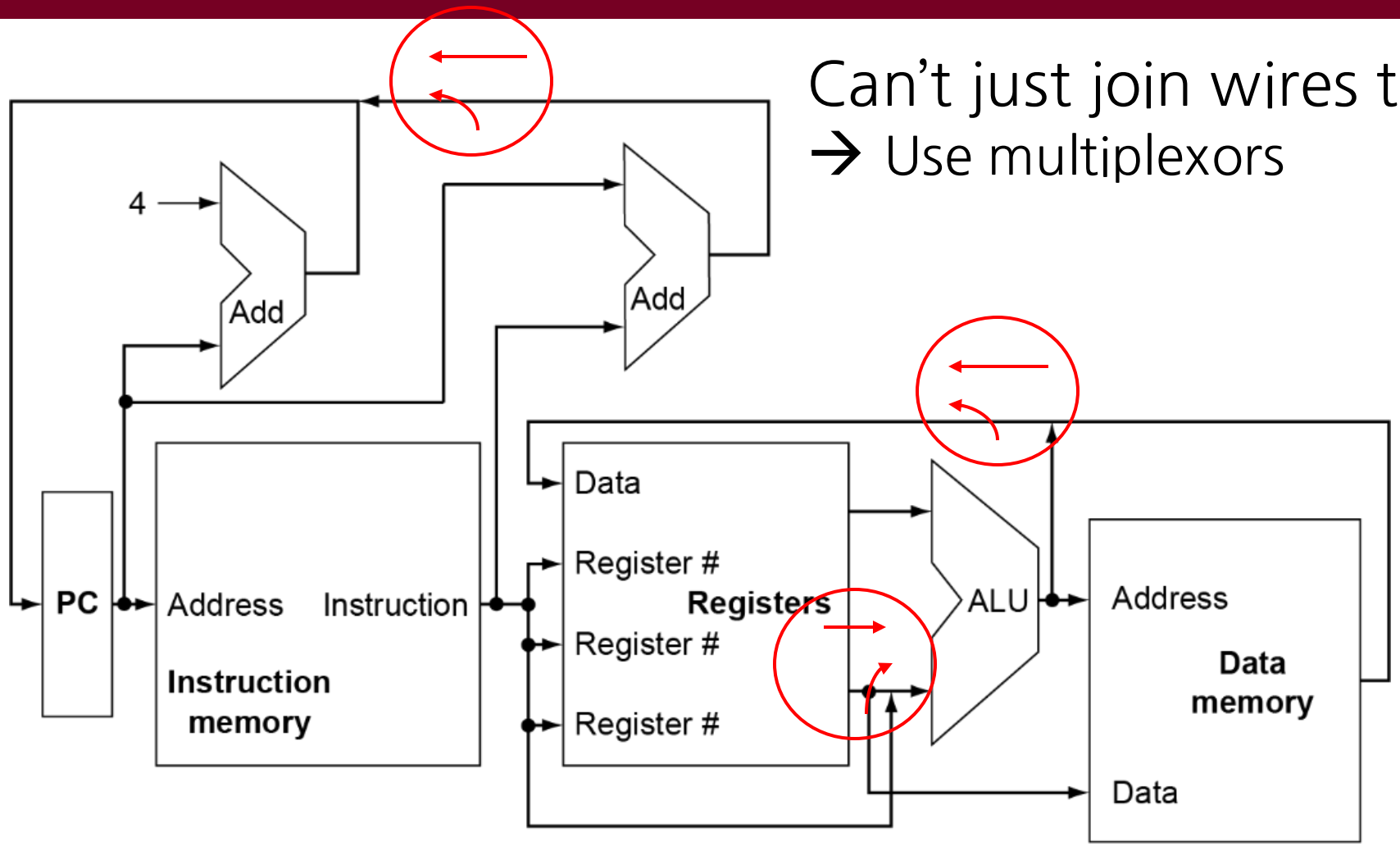
A Big Picture



CPU (RISC-V) Overview

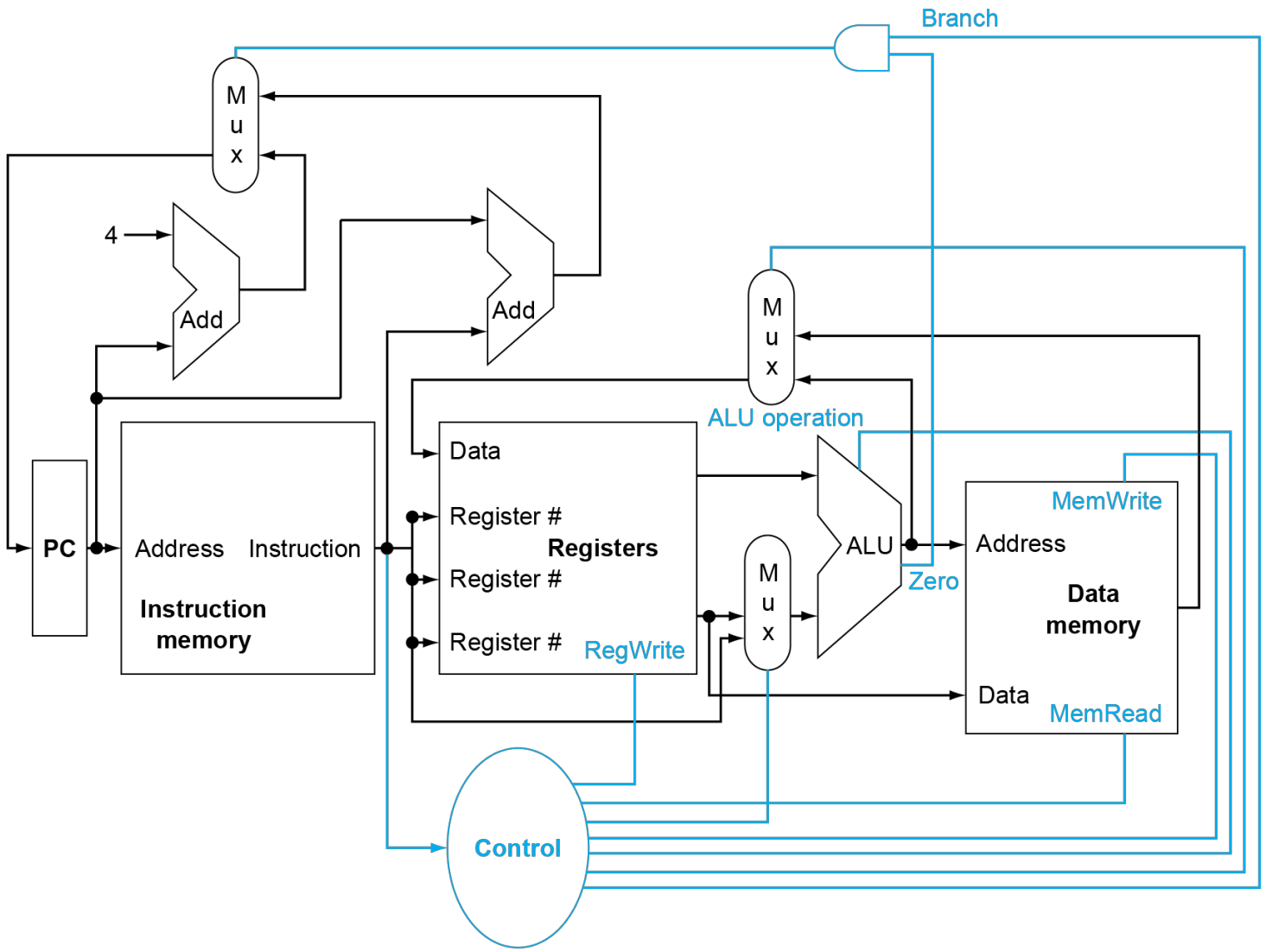


Multiplexers



Can't just join wires together
→ Use multiplexors

Control

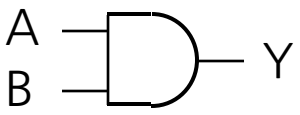

Branch
M
u
x
4
Add
Add
M
u
x
ALU operation
PC
Address
Instruction
Instruction
memory
Data
Register #
Registers
Register #
Register #
RegWrite
M
u
x
ALU
Zero
MemWrite
Address
Data
memory
Data
MemRead
Control

Logic Design Basics

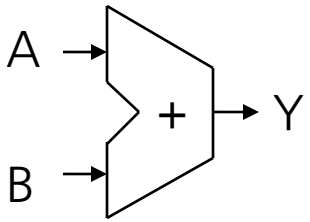
- Information encoded in binary
  - Low voltage = 0, High voltage = 1
  - One wire per bit
  - Multi-bit data encoded on multi-wire buses
- Combinational element
  - Operate on data
  - Output is a function of input
- State (sequential) elements
  - Store information

Combinational Elements

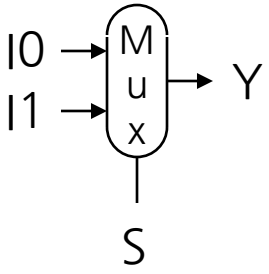
- AND-gate
  - Y = A & B



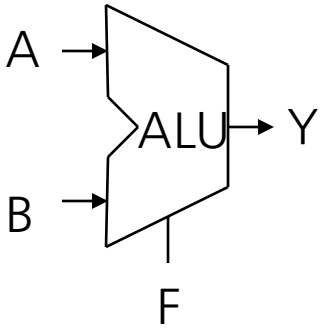
- Adder
  - Y = A + B



- Multiplexer
  - Y = S ? I1 : I0



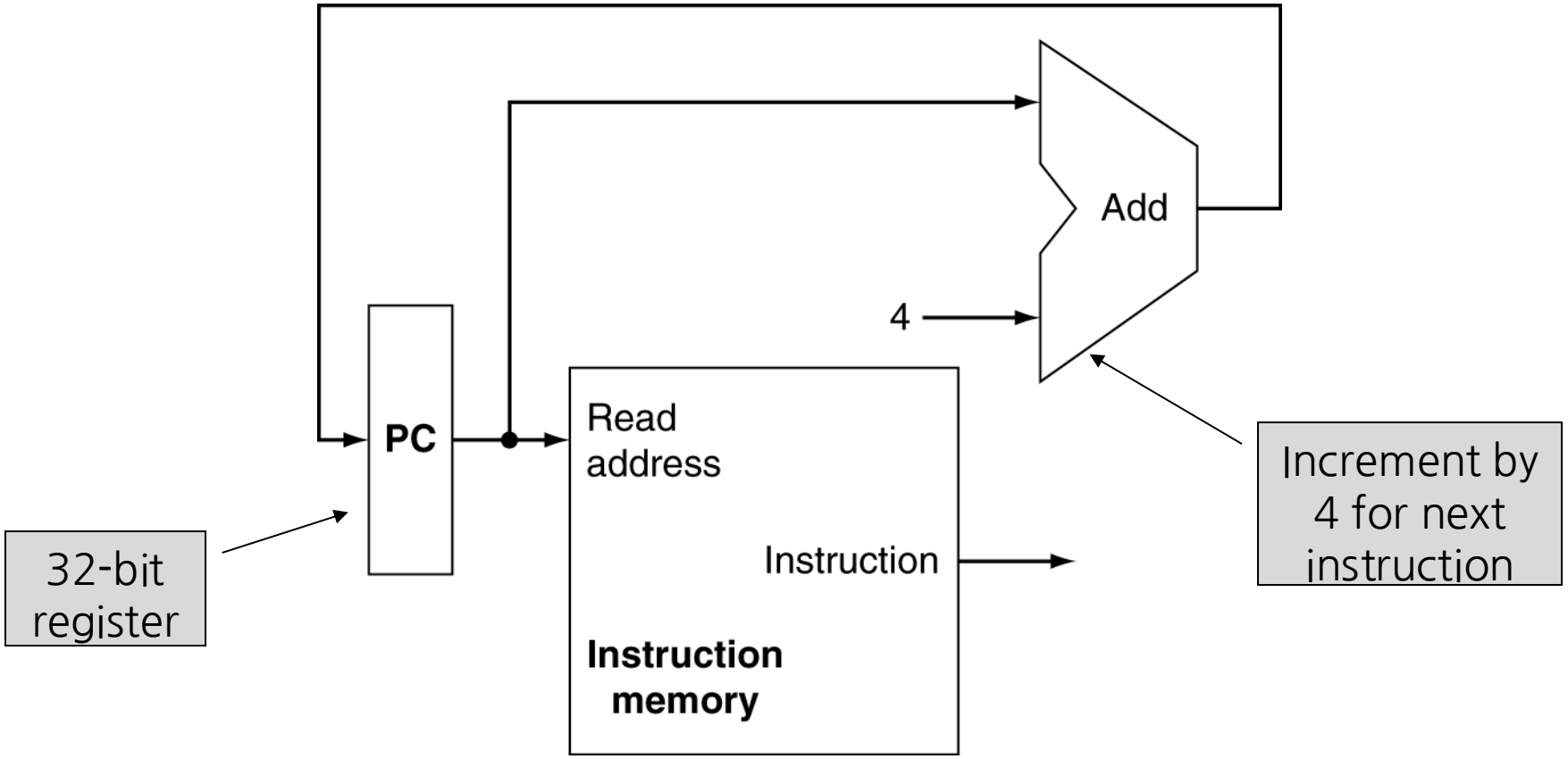
- Arithmetic/Logic Unit
  - Y = F(A, B)



Building a Datapath

- Datapath
  - Elements that process data and addresses in the CPU
    - Registers, ALUs, MUX's, memories, …
- We will build a RISC-V datapath incrementally
  - Refining the overview design

Instruction Fetch

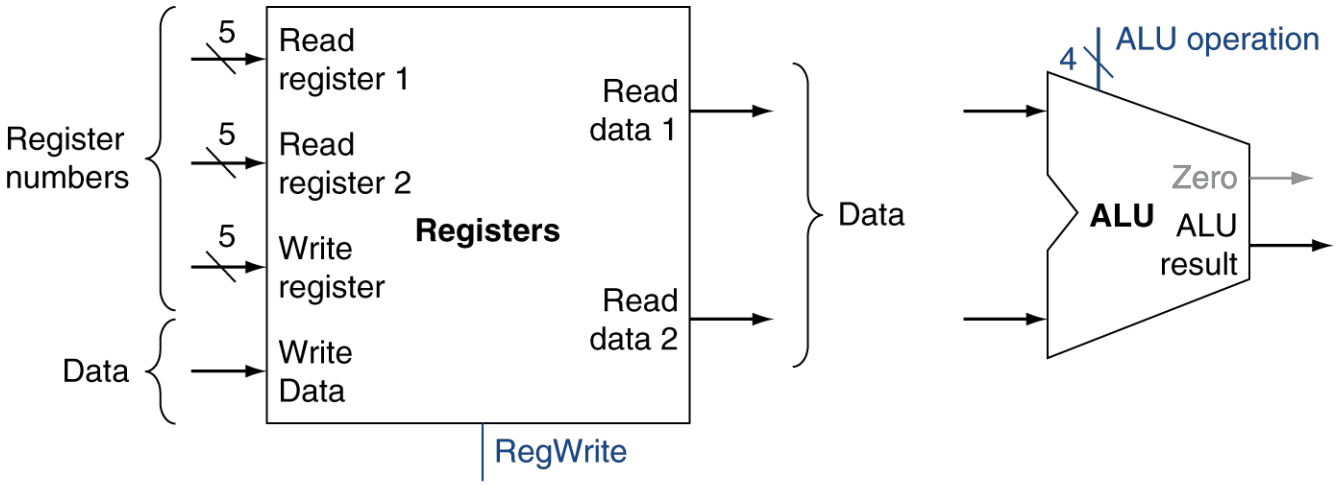


R-Format Instructions

- Read two register operands
- Perform arithmetic/logical operation
- Write register result

funct7	rs2	rs1	funct3	rd	opcode
7 bits	5 bits	5 bits	3 bits	5 bits	7 bits

`Instructions: add, sub, and, or`

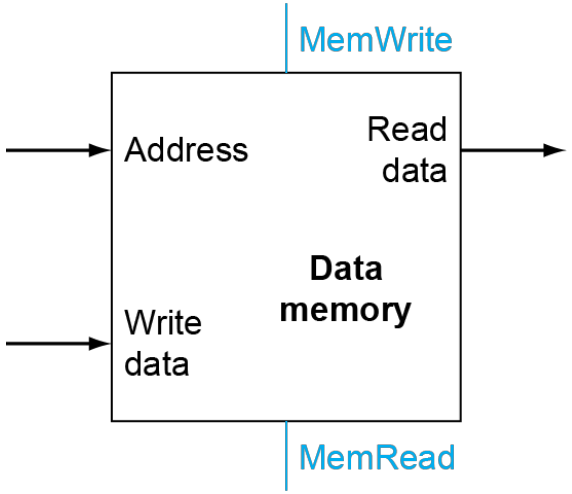


a. Registers

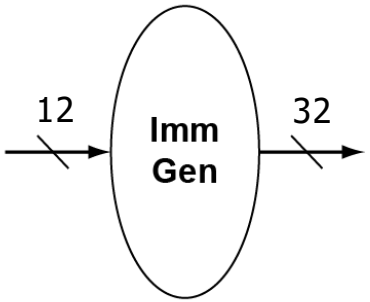
b. ALU

Load & Store Instructions

- Read register operand
- Calculate address using 12-bit offset
  - Use ALU, but sign-extended offset
- Load: Read memory and update register
- Store: Write register value to memory

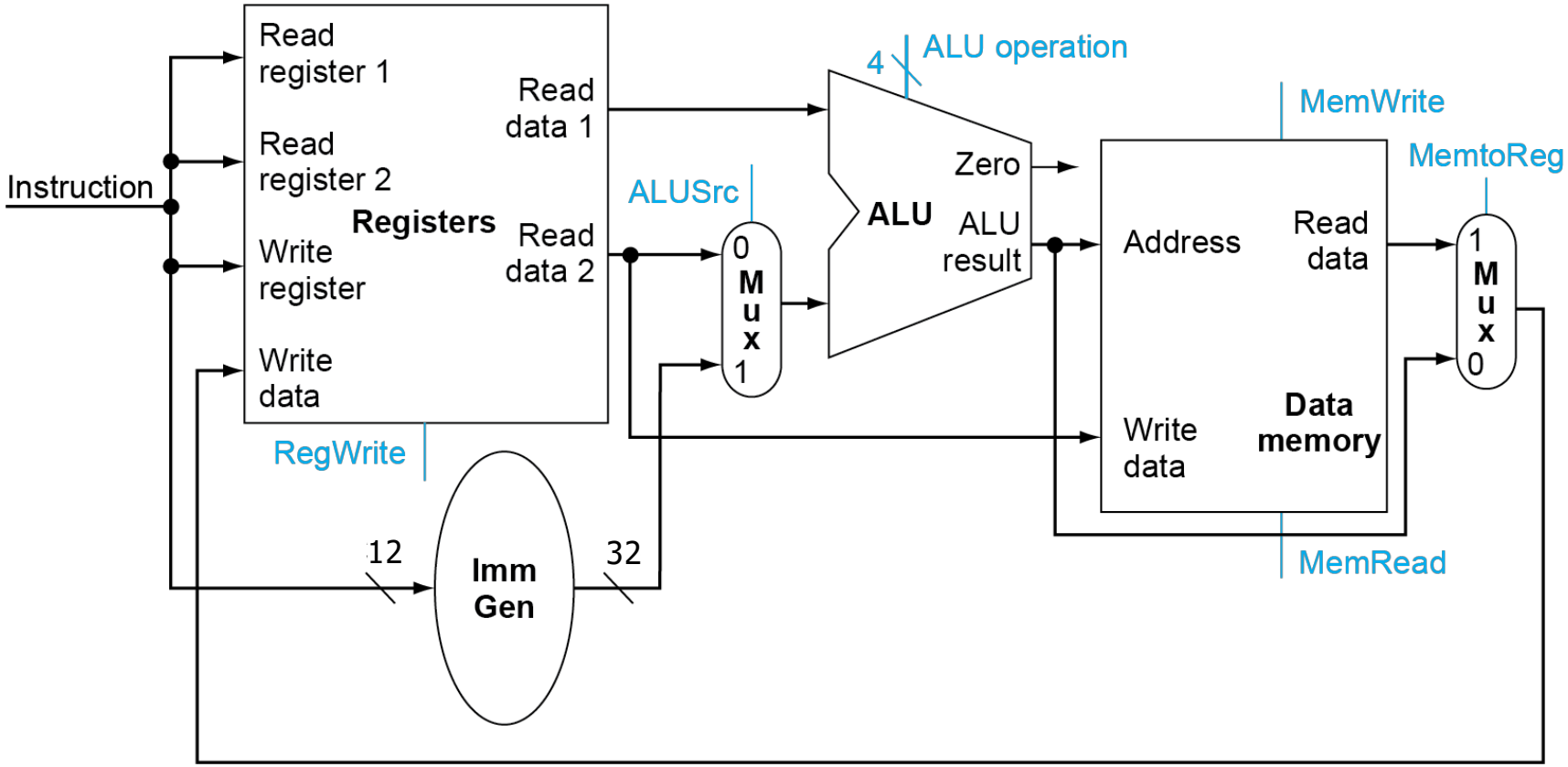


a. Data memory unit



b. Immediate generation unit

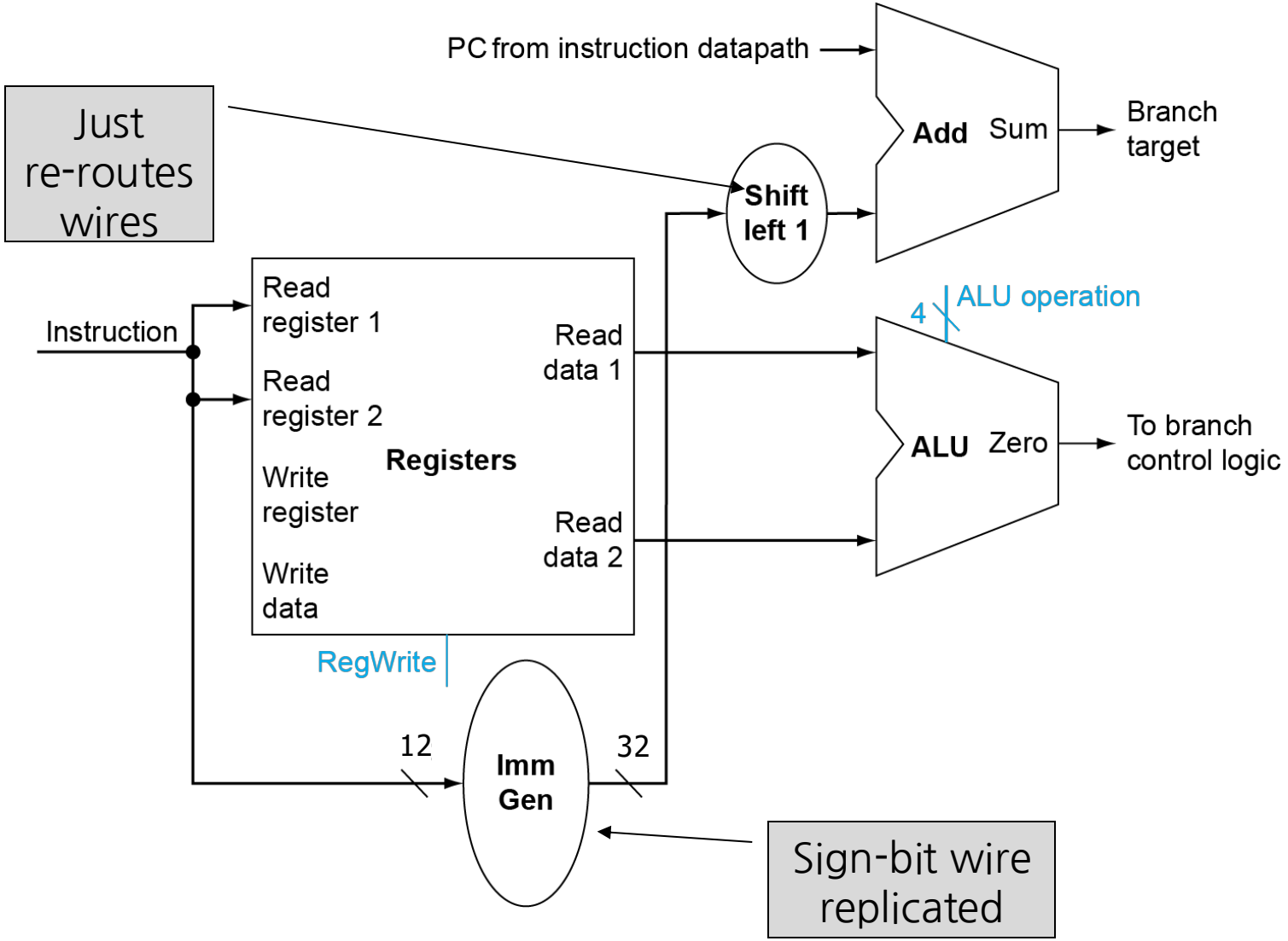
R-Type/Load/Store Datapath



Branch Instructions (1)

- Read register operands
- Compare operands
  - Use ALU, subtract and check Zero output
- Calculate target address
  - Sign-extend displacement
  - Shift left 1 places (word displacement)
  - Add to PC

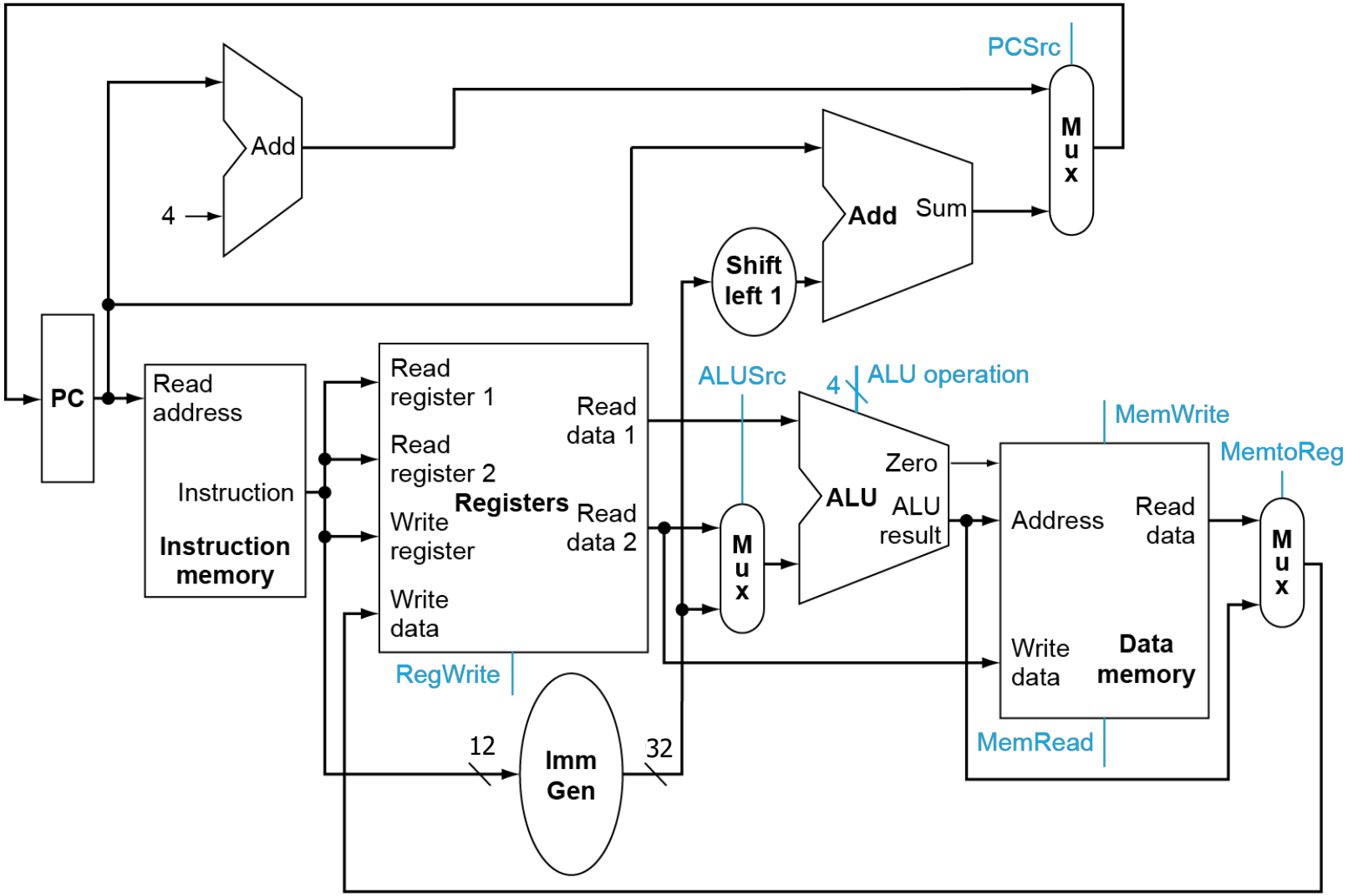
Branch Instructions (2)



Composing the Elements

- First-cut data path does an instruction in one clock cycle
  - Each datapath element can only do one function at a time
  - Hence, we need separate instruction and data memories
- Use multiplexers where alternate data sources are used for different instructions

Full Datapath



The Main Control Unit

- Control signals derived from instruction

Name (Bit position)	Fields 31:25	24:20	19:15	14:12	11:7	6:0
(a) R-type	funct7	rs2	rs1	funct3	rd	opcode
(b) I-type	immediate[11:0]		rs1	funct3	rd	opcode
(c) S-type	immed[11:5]	rs2	rs1	funct3	immed[4:0]	opcode
(d) SB-type	immed[12,10:5]	rs2	rs1	funct3	immed[4:1,11]	opcode

ALU Control (1)

- ALU used for
  - Load/Store: F = add
  - Branch: F = subtract
  - R-type: F depends on `opcode`

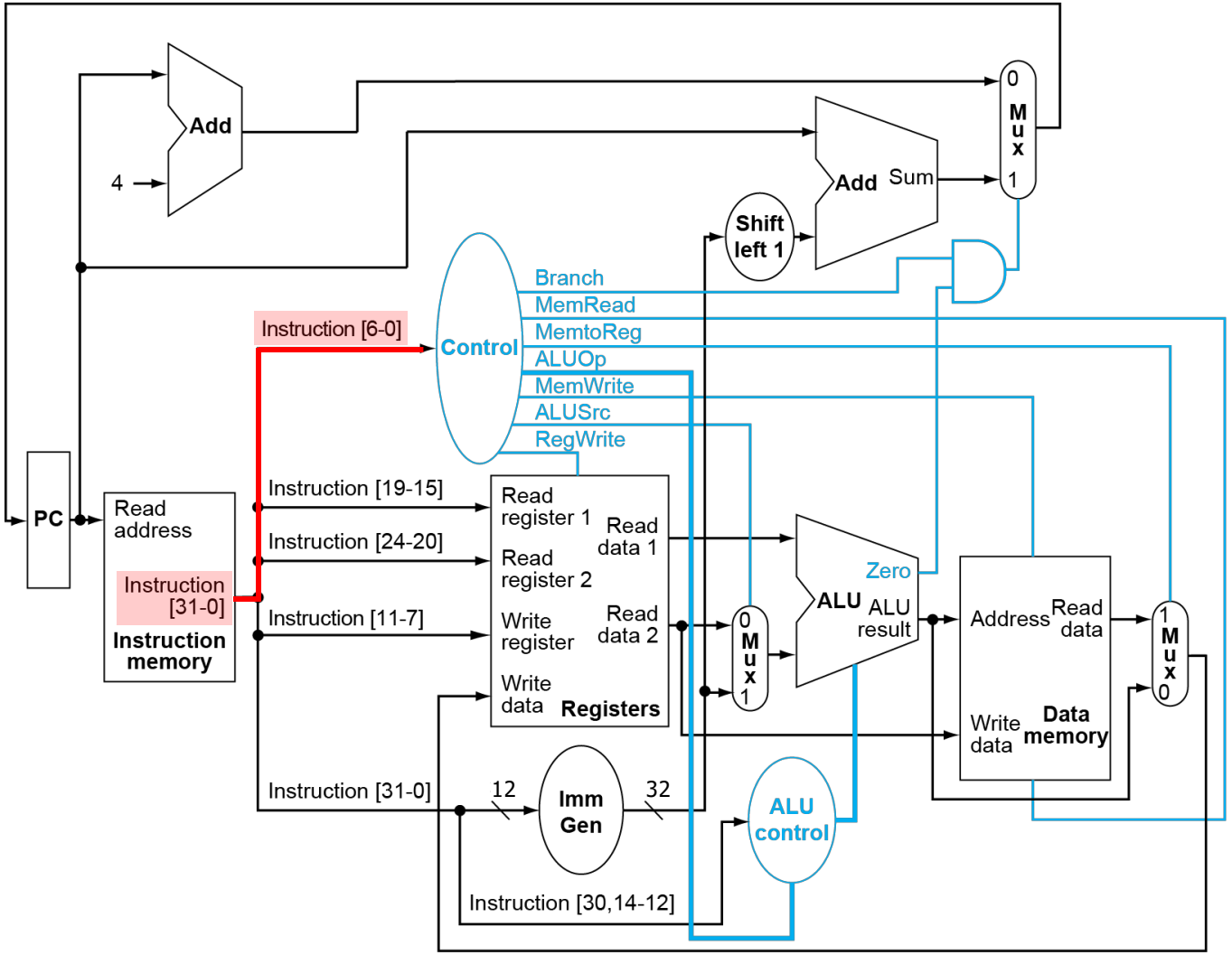
ALU control	Function
0000	AND
0001	OR
0010	add
0110	subtract

ALU Control (2)

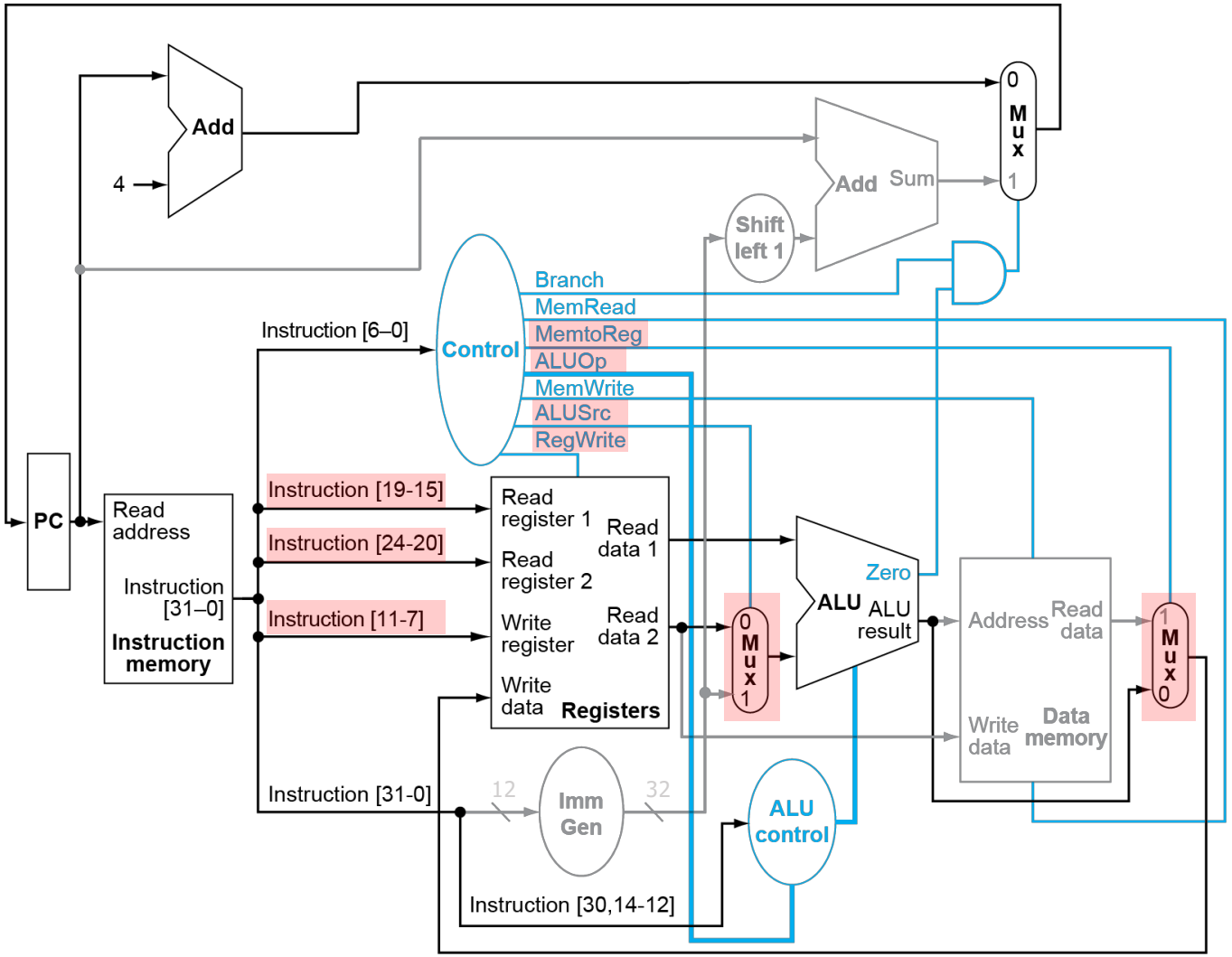
- Assume 2-bit ALUOp derived from opcode
  - Combinational logic derives ALU control

opcode	ALUOp	Operation	Opcode filed	ALU function	ALU control
ld	00	load word	XXXXXX	add	0010
sd	00	store word	XXXXXX	add	0010
beq	01	branch equal	XXXXXX	subtract	0110
R-type	10	add	100000	add	0010
		subtract	100010	subtract	0110
		AND	100100	AND	0000
		OR	100101	OR	0001

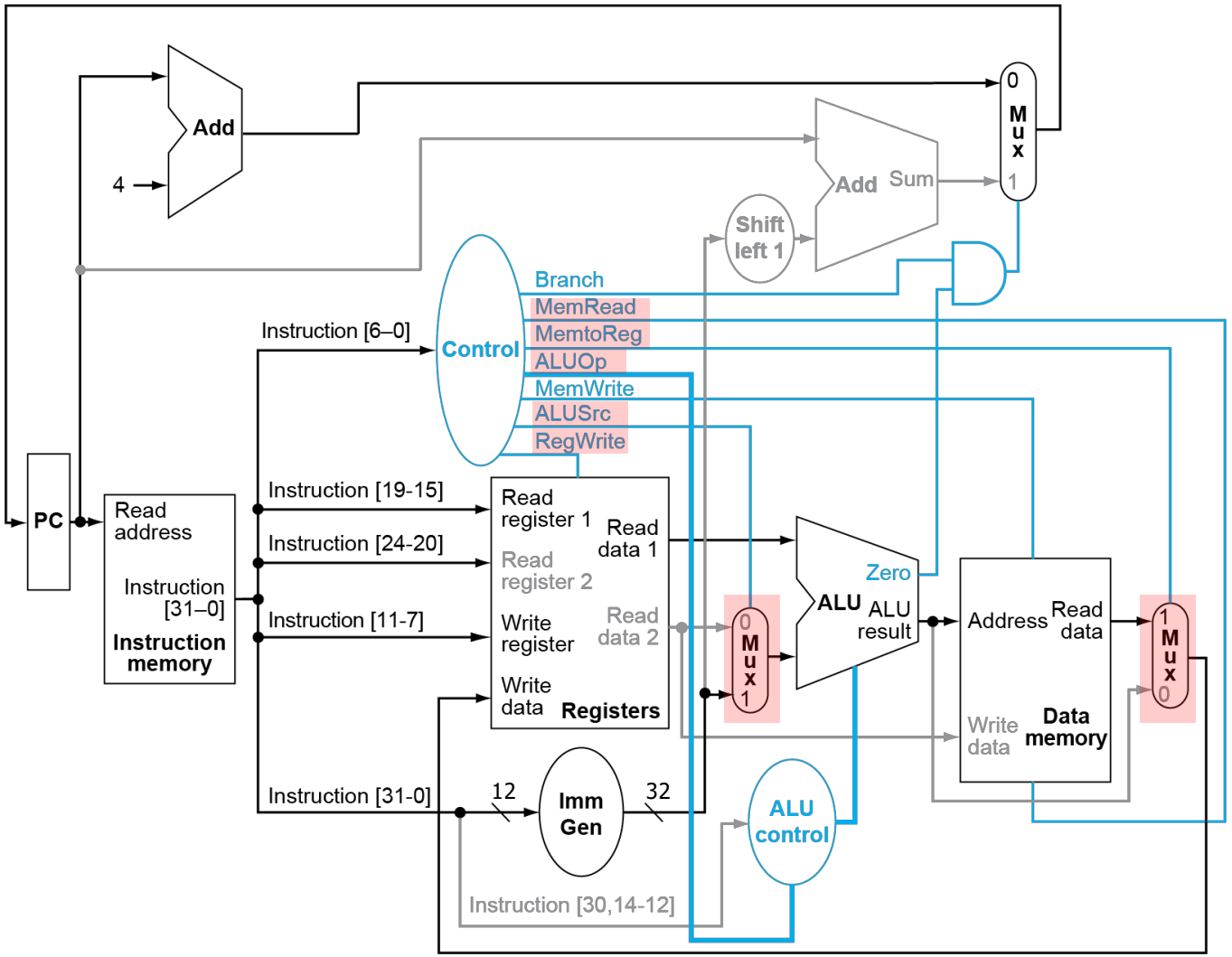
Datapath with Control



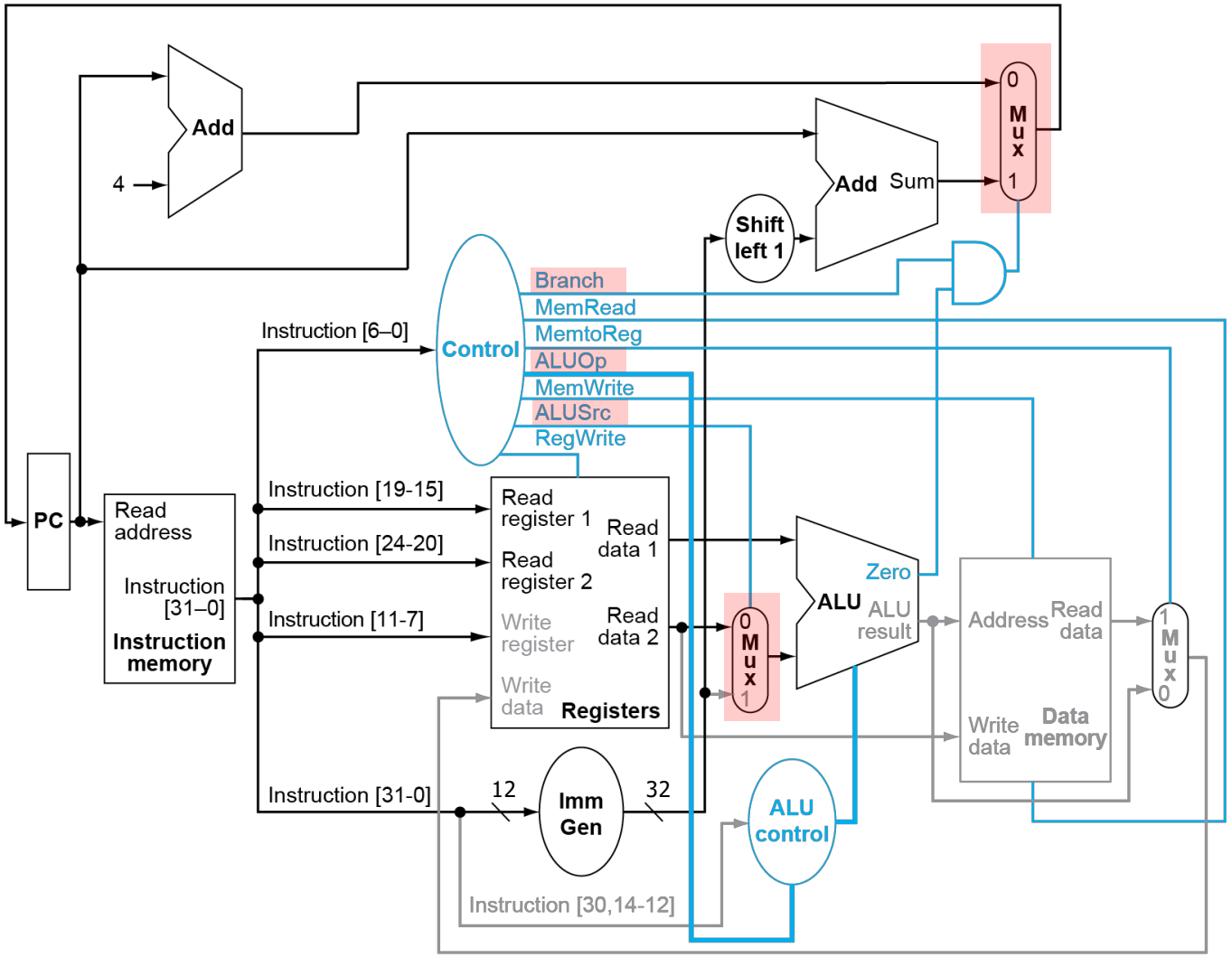
R-Type Instruction



Load Instruction



Branch-on-Equal Instruction



Performance Issues

- Longest delay determines clock period
  - Critical path: load instruction
  - Instruction memory → register file → ALU → data memory → register file
- Not feasible to vary period for different instructions
- Violates design principle
  - Making the common case fast
- We will improve performance by pipelining